

Управление сервоприводом *MG996R* с помощью платы *STM32F103* и программы *PicOS-MG996R*

Дата: 2025.10.10

Автор: Шендрик Павел Дмитриевич

Оглавление

Введение	1
Схема подключения	3
Описание программы	4
Работа программы	6
Список литературы.....	9

Введение

Для изменения расстояния с помощью платы *STM32F103 BluePill*, также дополнительно необходимы: сервопривод *MG996R* (или любой другой на основе *SG90*), программатор *STLINK V2*, преобразователь *USB-TO-UART* и программа *PicOS-MG996R*. Для взаимодействия с программой рекомендуется использовать *Terminal v1.9b*.

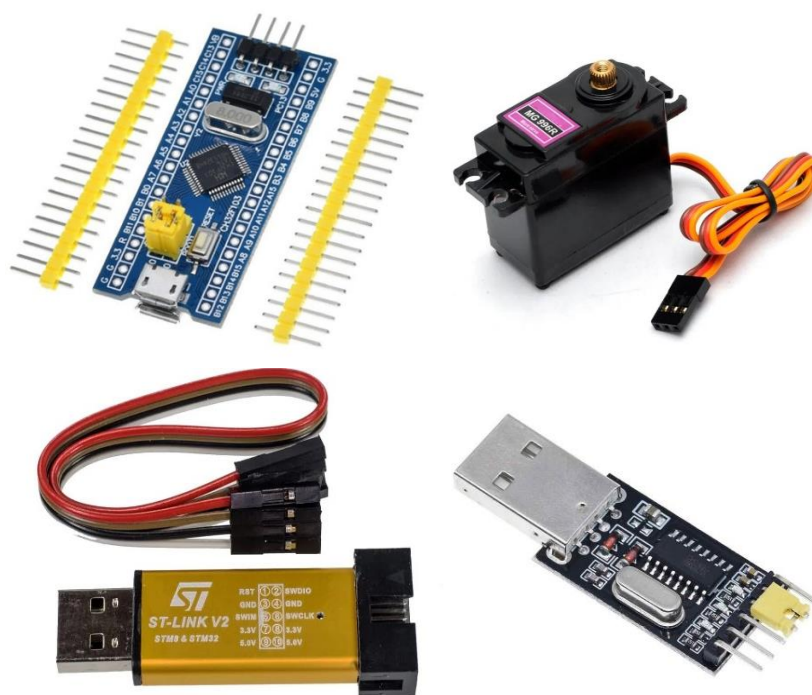


Рис. 1 *STM32F103 BluePill*, *MG996R*, *ST-LINK V2* и *CH340* (*USB-TO-UART* преобразователь).

Для управления сервоприводом необходимо подавать сигнал соответствующей длительности с периодом в 20 мс (рис. 2). Поэтому необходимо использовать ШИМ, для

корректной работы которого, необходимо настроить таймер (рис. 3). Более подробно о работе сервопривода можно прочитать в следующем источнике [1].

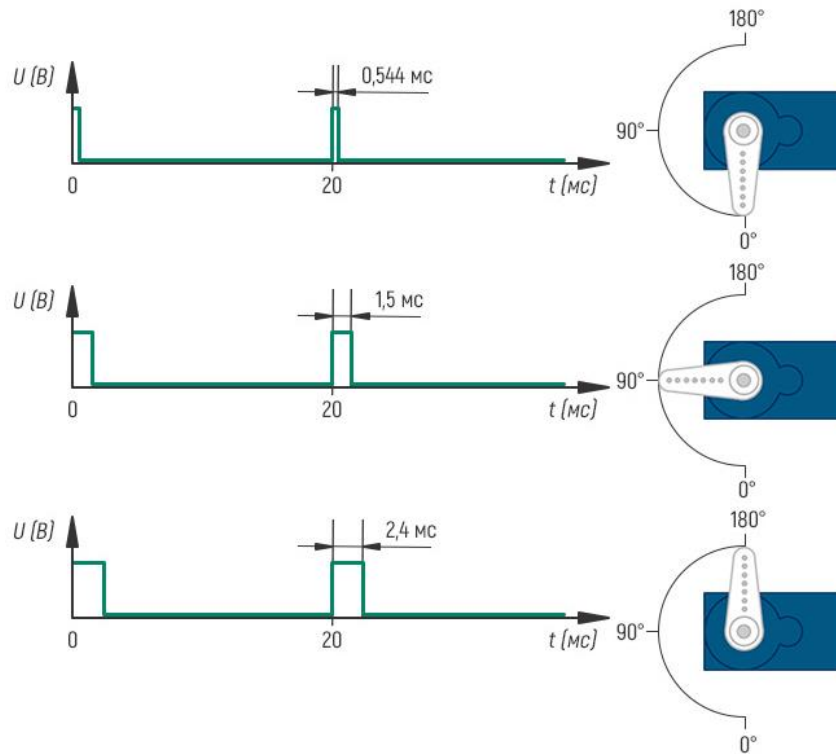


Рис. 2 Управление сервоприводом

```
// ===== TIM2 =====
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN; // Разрешаем тактирование, PCLK1 = 72 МГц
TIM2->SMCR &= ~TIM_SMCR_SMS; // Выбираем внутреннее тактирование
TIM2->PSC = 72 - 1; // Шаг (квант) счёта 1 мкс
TIM2->ARR = 40000; // Переполнение каждые 20 мс (50 Гц)
TIM2->CCMR2 &= ~TIM_CCMR2_CC4S; // Выбор выхода TIM2_CH4
TIM2->CCMR2 |= (TIM_CCMR2_OC4M_2 | TIM_CCMR2_OC4M_1); // Режим PWM
TIM2->CCER |= TIM_CCER_CC4E; // Включение выхода канала TIM2_CH4 на порт
TIM2->CCR4 = 3000; // По умолчанию импульс 1 мс
TIM2->CR1 |= TIM_CR1_CEN; // Включение таймера-счетчика
}
```

Рис. 3 Настройка таймера 2

В настройке таймера стоит обратить внимание на переполнение каждые 20 мс (как на рис. 2), режим работы – ШИМ и настройку вывода четвёртого канала таймера на порт. Так как шаг счёта настроен на 1 мкс, управление сервоприводом будет находиться, примерно, в интервале от 544 до 2400 мкс, как на рис. 2.

Схема подключения

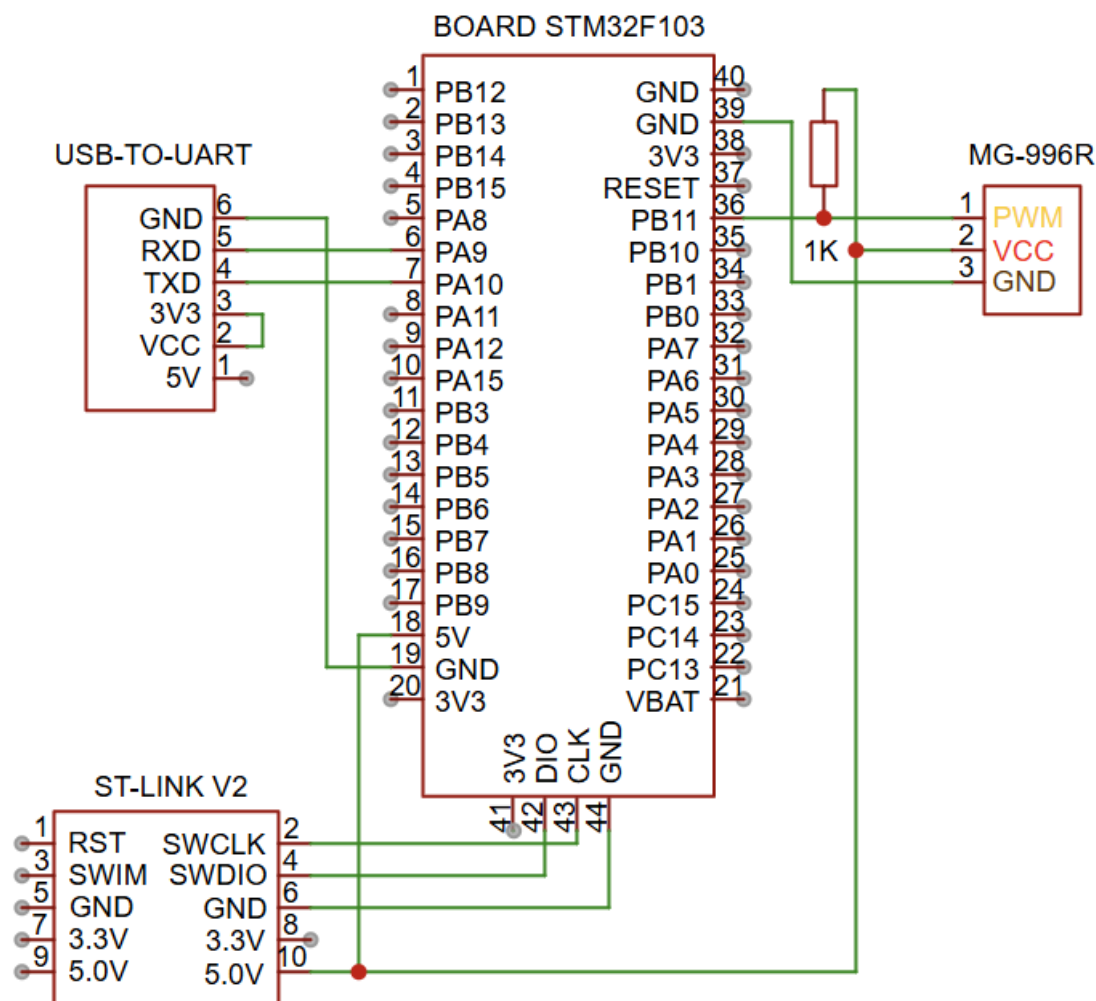


Рис. 4 Схема подключения

Названия портов 41 – 44, могут отличаться от схемы, поэтому необходимо подключать контакты *ST-LINK V2* к соответствующим контактам платы *STM32F103*: *SWCLK* к *SWCLK* (или его сокращению *CLK*, *SWK*, и т.п.); *SWDIO* к *SWDIO* (или его сокращению *DIO*, *D* и т.п.); *GND* к *GND* (или его сокращению *G* и т.п.); *5V* к *5V*.

На сервоприводе отсутствуют пометки с названиями контактов, поэтому оранжевый провод для управляющего сигнала *PWM* подключается к контакту платы *PB11*, также подключаясь к питанию «5V» через резистор номиналом 1 кОм, для повышения напряжения линии с 3.3 В до 5 В. Красный провод *VCC* отвечает за питание сервопривода, поэтому к нему подключается контакт с питающим напряжением 5V, также соединяющийся с резистором. Коричневый провод *GND* подключается к соответствующему контакту платы с пометкой *GND*.

Описание программы

PicOS – простейшая операционная система (далее «ОС»), обеспечивающая минимальный набор базовых функций. Программа состоит из конечных автоматов [2, 3], постоянно взаимодействующих друг с другом. Главной задачей ОС является общение между системой и пользователем: обработка полученных строк, ответ на команды, вывод и изменение переменных.

В главном цикле *while* (рис. 5) находятся вставки *#include* с содержимым всех автоматов. Такой способ представления информации используется для экономии места, улучшения читаемости главного цикла и ускорения работы системы – во время компиляции программы, вставка заменяется на соответствующий файл, что приравнивается к копированию всего содержимого автомата в *while*, а использование функций, вместо вставок сказывается на быстродействии (т.к. используются вызовы и возвраты из функций), что очень критично для встраиваемых систем. Все автоматы находятся в папке «Tasks», они содержат собственное краткое описание и комментарии к коду.

```
// ===== ГЛАВНЫЙ ЦИКЛ =====  
while (1) {  
    #include "Tasks/usart1_send.c" // Отправка данных по USART1  
    #include "Tasks/usart1_recv.c" // Принятие строки по USART1  
    #include "Tasks/picos.c" // Реализация PicOS  
    #include "Tasks/var_get.c" // Вывод значений переменных по USART1  
    #include "Tasks/var_set.c" // Изменение значений переменных  
    #include "Tasks/led_switch.c" // Изменение состояния светодиода PC13  
    #include "Tasks/servo_drive_switch.c" // Изменение положения сервопривода  
}
```

Рис. 5 Главный цикл программы

Все автоматы можно условно разделить на системные, главные и пользовательские:

- Системные автоматы (*usart1_recv*, *usart1_send*, *var_get*, *var_set*) выполняют функции операционной системы: обработку ввода/вывода, управление переменными.
- Главный автомат (*picos*) опрашивает и управляет работой других автоматов, а также анализирует полученные команды.
- Пользовательские автоматы (*led_switch*, *servo_drive_switch*) реализуют конкретные функции управления или опрашивания по пользовательским командам.

Таким образом, шестой автомат служит для включения и выключения встроенного светодиода *PC13*. Включение светодиода произойдёт после получения команд «led 1» или «led = 1», а выключение – после получения «led 0» или «led = 0». А седьмой нужен для управления сервоприводом *MG996R* за счёт изменения переменной *servo*, получающей желаемый угол поворота сервопривода от 0° до 180°. Все команды изменяют или запрашивают значения переменных (*led*, *servo* и т.п.), которые и отслеживаются в программе.

Все переменные, которые можно изменить или запросить командой, описываются в таблице переменных *var_table[]* (рис. 6), каждый элемент таблицы представляет собой структуру из четырёх элементов: Отображаемое имя переменной *name[12]*, краткое описание переменной *description[32]*, тип переменной *type* и указатель на саму

переменную **ptr*. Таким образом таблица из переменных *version*, *led* и *servo* будет выглядеть так:

```
// ===== Инициализация таблицы переменных =====  
var_table_t var_table[] = {  
    {"version", "Version of the firmware", TYPE_UINT16_T, &fm_version},  
    {"led", "LED PC13 state: 1 - ON, 0 - OFF", TYPE_UINT16_T, &led_status},  
    {"servo", "Angle of rotation: 0..180", TYPE_UINT16_T, &servo_drive_switch}  
};
```

Рис. 6 Таблица переменных *var_table[]*

Где, *version* – версия программы (равна 33, используется для преобразования в 3.3);

led – статус светодиода: ≥ 1 – включен, 0 – выключен;

servo – желаемый угол поворота сервопривода: от 0° до 180°;

Таблицу переменных *var_table[]* можно найти в файле «main.c» или вывести командой «help».

Работа программы

После запуска программы автоматы *usart1_send* и *usart1_recv* бездействуют, т.к. их состояния «*usart1_..._state*» равны «*USART1_..._NOT_ACTIVATED*». Первым запустившимся автоматом будет *picos*, который запустит *usart1_send*, для вывода приветственного сообщения, и *usart1_recv*, для получения команд пользователя. По умолчанию состояния автоматов *var_get* и *var_set* также равны «*..._NOT_ACTIVATED*», и они будут неактивны. Их работа регулируется автоматом *picos*, после получения соответствующих команд от пользователя.

Например, после получения автоматом *usart1_recv* команды «*led 1*» и её последующей обработки автоматом *picos*, он запустит автоматы *var_set*, который изменит значение переменной, и *var_get*, который и выведет изменённую переменную с её новым значением. Из-за изменения переменной *led*, сработает автомат *led_switch* и включит встроенный светодиод *PC13*.

На самом деле программа сложнее, т.к. обработка строки происходит за несколько циклов программы, а светодиод включится раньше, чем пользователь получит строку с новым значением переменной.

Автомат *servo_drive_switch* постоянно работает и на каждом цикле программы записывает новое желаемое значение угла поворота сервопривода в четвёртый канал таймера 2 (*TIM2->CCR4*), что приводит к изменению сигнала ШИМ на *PB11*.

Для запуска программы:

1. Соберите схему (рис. 7);
2. Вставьте *CH340* и *ST-LINK V2* в компьютер и подключитесь к новому *COM*-порту в приложении *Terminal*;
3. Перепрограммируйте МК запустив *_comp+prog.bat*, чтобы в *Terminal* вывелось приветственное сообщение «Welcome to PicOS v3.3».

Для введения команд рекомендуется использовать окно посимвольного ввода в программе *Terminal* (самое нижнее). Все введённые команды требуют подтверждение – получение системой специального управляющего символа «+CR (Carriage Return)», который можно отправить нажатием на *Enter*, поэтому команду до подтверждения можно исправлять. Корректная работа программы и настройки *Terminal* изображены на рисунке 8.

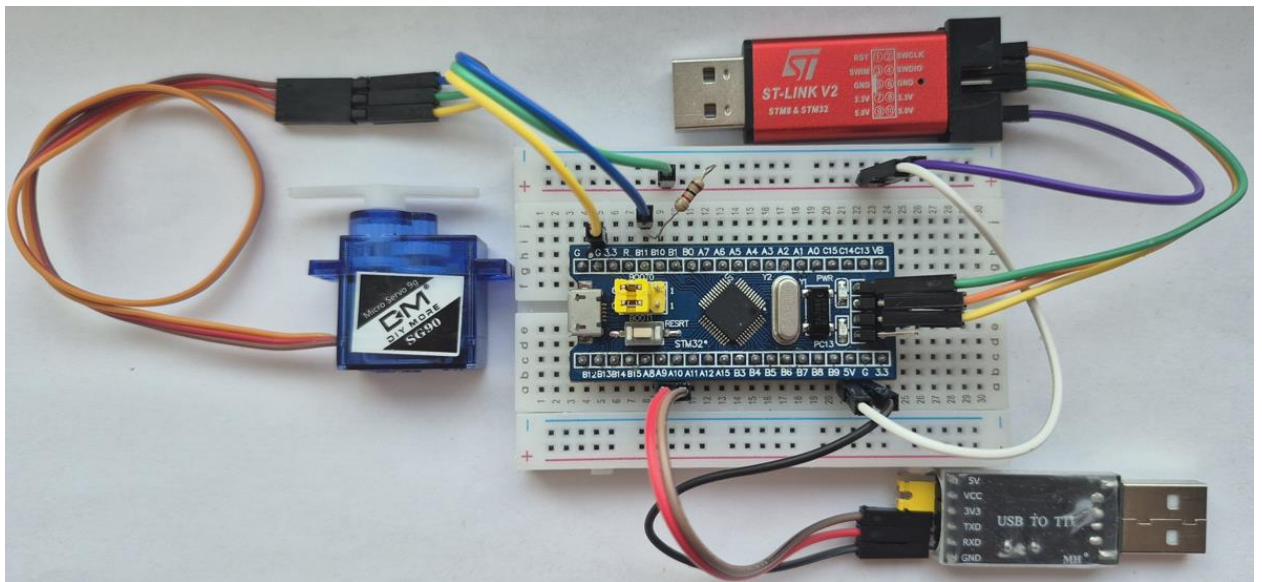


Рис. 7 Собранная схема

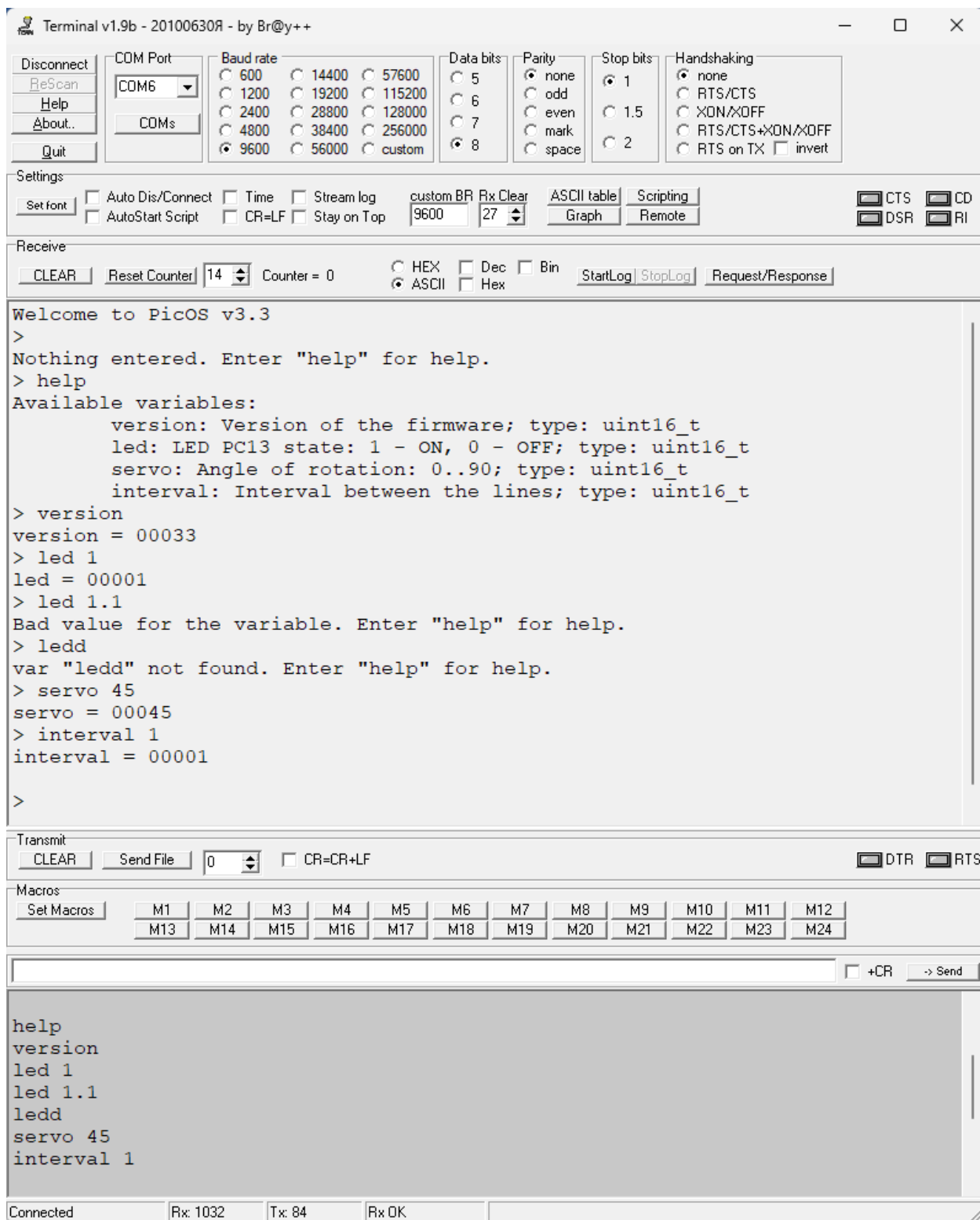


Рис. 8 Результаты работы программы *PicOS-MG996R* и настройки *Terminal*

Список литературы

1. Сервопривод / AmperMarket // URL: <https://ampermarket.kz/base/servomotors/> (дата обращения: 13.10.2025).
2. Введение в автоматное программирование микроконтроллеров STM32 / В.А. Вдовин, И.В. Трубин, В.Г. Трубин, П.Д. Шендрик // Известия Томского политехнического университета. Промышленная кибернетика. –2024. –Т. 2. –No 4. –С. 21–25. DOI:10.18799/29495407/2024/4/75. URL: <https://indcyb.ru/journal/article/view/75/60> (дата обращения: 01.02.2025).
3. Пример псевдопараллельного выполнения трёх задач на микроконтроллере STM32F103 без использования операционной системы / В.А. Вдовин, И.В. Трубин, В.Г. Трубин, П.Д. Шендрик // Известия Томского политехнического университета. Промышленная кибернетика. –2025. –Т. 3. –No 2. –С. 1–7. DOI:10.18799/29495407/2025/2/88 // URL: <https://indcyb.ru/journal/article/view/88/71> (дата обращения: 30.06.2025).